



UNIVERSIDAD ARGENTINA DE LA EMPRESA
Facultad de Ingeniería y Ciencias Exactas - FAIN

Materia: PROCESO DE DESARROLLO DE SOFTWARE

Trabajo Práctico Obligatorio – Proceso de Desarrollo de Software

Fase 2: Diseño orientado a objetos y patrones

03-06-2026 - Buenos Aires, Argentina

Integrantes del grupo:

Cortés, Ignacio 1203485
Etchart, Facundo 1201391
Rubachin, Sofía 1196599

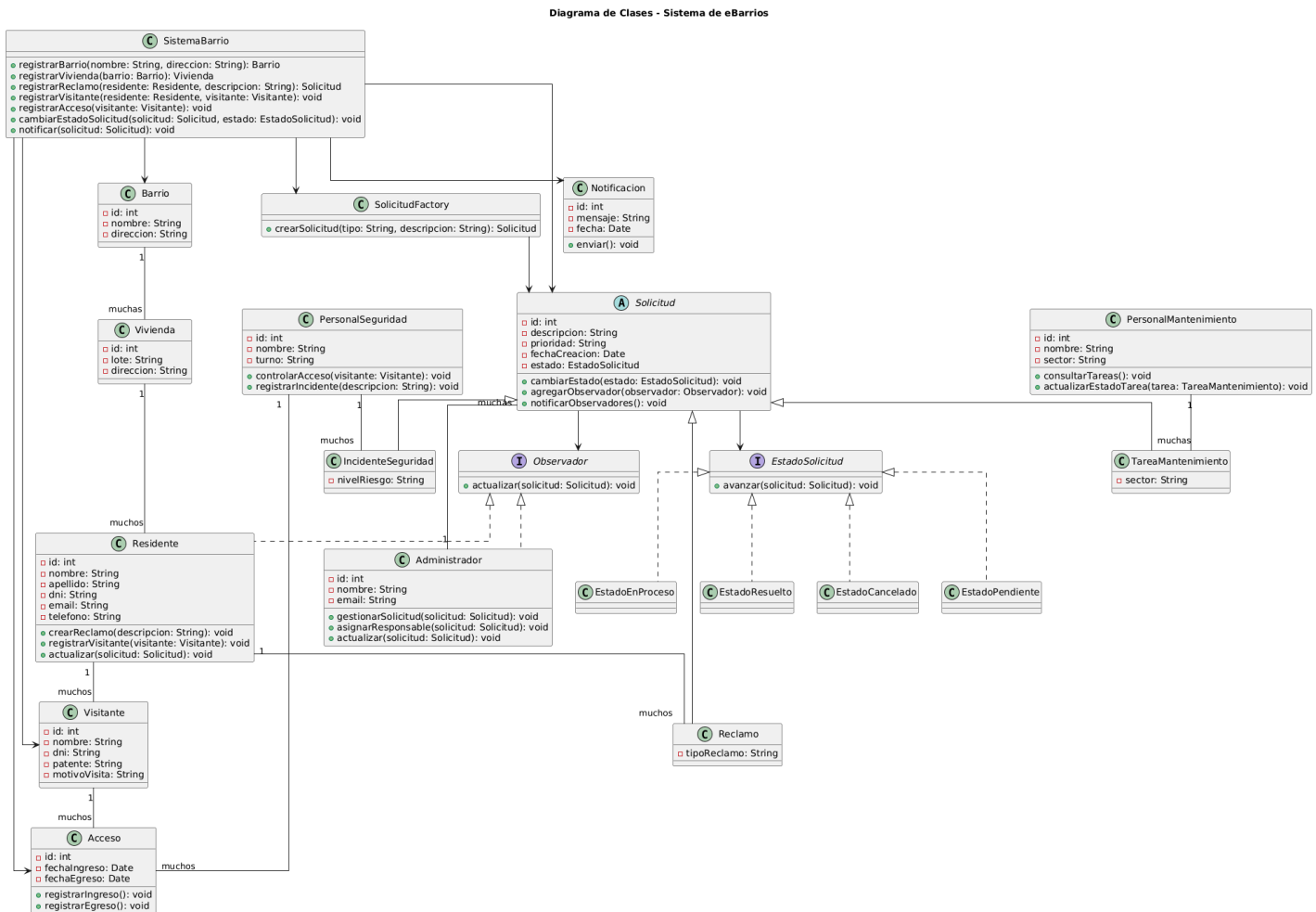
Profesor:

Ing. León María Ángela

Índice

Índice.....	1
1. Diagrama de clases.....	2
2. Diagramas de secuencias.....	3
1. Crear reclamo.....	3
2. Cambiar estado de solicitud.....	3
3. Identificación de patrones de diseño a aplicar.....	4
Factory Method / Simple Factory.....	4
State.....	4
Observer.....	4
Facade / Controller.....	4
4. Justificación de cada patron seleccionado.....	4
Factory Method / Simple Factory.....	4
State.....	5
Observer.....	5
Facade / Controller.....	5
5. Identificación de principios SOLID aplicados.....	5
Single Responsibility Principle.....	5
Open/Closed Principle.....	5
Liskov Substitution Principle.....	5
Interface Segregation Principle.....	5
Dependency Inversion Principle.....	5
6. Identificación de patrones GRASP aplicados.....	6
Information Expert.....	6
Creator.....	6
Controller.....	6
Low Coupling.....	6
High Cohesion.....	6
Polymorphism.....	6
7. Estructura inicial de paquetes del proyecto Java.....	6
8. Primera versión del código con clases principales, interfaces y relaciones.....	7
9. Informe breve de avance individual por integrante.....	8
Etchart, Facundo.....	8
Cortés, Ignacio.....	8
Rubachin, Sofía.....	8
Anexo.....	9
• Diagrama de clases en PlantUML.....	9
• Diagrama de casos de uso en PlantUML.....	13

1. Diagrama de clases



El diseño propuesto aplica principios GRASP y SOLID mediante una separación clara de responsabilidades. Cada clase representa un concepto específico del sistema y se evita concentrar toda la lógica en una única clase.

Se aplica **Information Expert** porque cada clase administra los datos que le corresponden, como **Solicitud** con su estado y prioridad o **Acceso** con los ingresos y egresos. También se aplica **Creator** mediante **SolicitudFactory**, encargada de crear los distintos tipos de solicitudes. La clase **SistemaBarrio** funciona como **Controller** y **Facade**, ya que coordina las operaciones principales del sistema y ofrece una interfaz simple para registrar reclamos, accesos y cambios de estado.

En cuanto a SOLID, el diseño respeta la responsabilidad única, ya que cada clase tiene una función concreta. También permite extender el sistema agregando nuevos tipos de solicitudes o nuevos estados sin modificar toda la estructura. Las clases **Reclamo**, **TareaMantenimiento** e **IncidenteSeguridad** pueden utilizarse como **Solicitud**, cumpliendo sustitución de Liskov. Además, se utiliza la interfaz **Observador** para notificar cambios, reduciendo el acoplamiento entre clases.

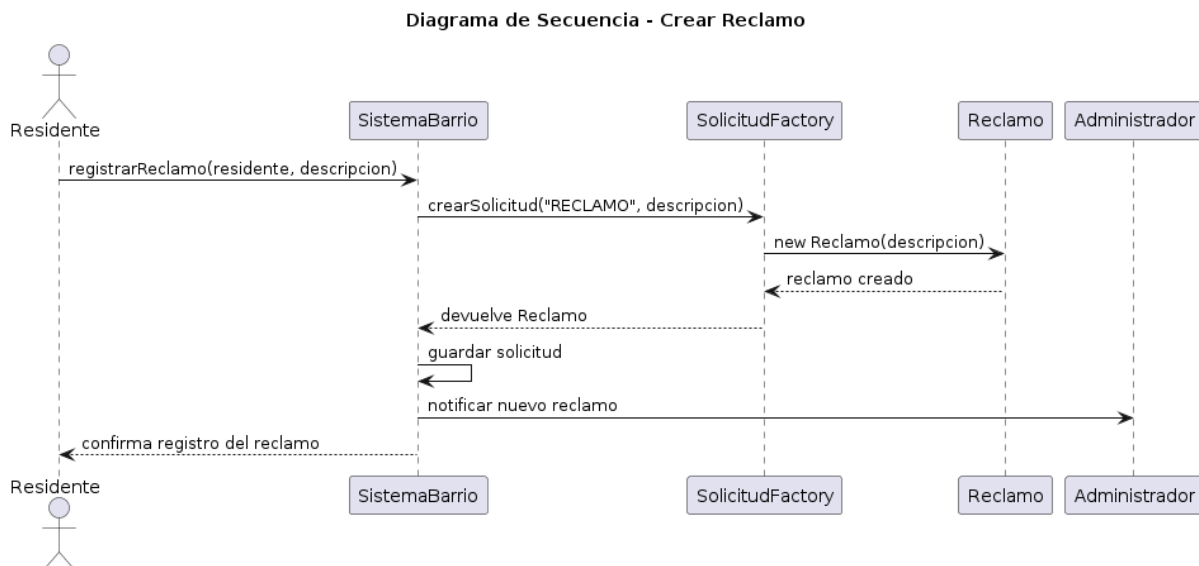
Los patrones aplicados son **Factory Method**, para crear solicitudes; **Observer**, para notificar cambios de estado; **State**, para representar los estados de una solicitud; y **Facade**, mediante la clase SistemaBarrio, que simplifica el acceso a las operaciones principales.

2. Diagramas de secuencias

1. Crear reclamo

Este diagrama muestra el proceso de creación de un reclamo. El residente solicita registrar un reclamo al sistema. Luego, SistemaBarrio utiliza SolicitudFactory para crear una solicitud de tipo Reclamo. Una vez creada, la solicitud queda registrada en el sistema y se notifica al administrador para que pueda gestionarla.

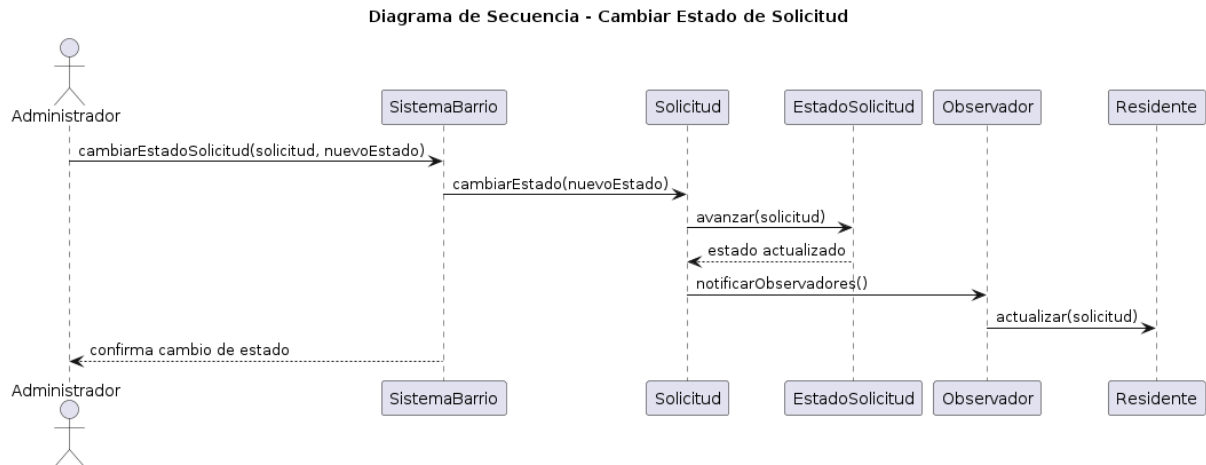
En este caso se aplica el patrón Factory Method / Simple Factory, ya que la creación del reclamo se centraliza en SolicitudFactory.



2. Cambiar estado de solicitud

Este diagrama representa el cambio de estado de una solicitud. El administrador solicita modificar el estado de una solicitud y SistemaBarrio delega esa acción en la clase Solicitud. La solicitud actualiza su estado mediante la interfaz EstadoSolicitud y luego notifica a sus observadores.

En este caso se aplican dos patrones de comportamiento: State, para representar los distintos estados de una solicitud, y Observer, para notificar automáticamente a los usuarios interesados cuando ocurre un cambio.



3. Identificación de patrones de diseño a aplicar.

Factory Method / Simple Factory

Se aplica mediante la clase SolicitudFactory, encargada de crear los distintos tipos de solicitudes del sistema. Esto permite centralizar la creación de objetos como Reclamo, TareaMantenimiento e IncidenteSeguridad.

State

Se aplica mediante la interfaz EstadoSolicitud y las clases concretas EstadoPendiente, EstadoEnProceso, EstadoResuelto y EstadoCancelado. Este patrón permite representar el estado actual de una solicitud y modificar su comportamiento según dicho estado.

Observer

Se aplica mediante la interfaz Observador, implementada por clases como Residente y Administrador. Permite que los usuarios sean notificados cuando una solicitud cambia de estado.

Facade / Controller

Se aplica mediante la clase SistemaBarrio, que centraliza las operaciones principales del sistema. Esta clase permite registrar reclamos, visitantes, accesos y cambios de estado sin que las demás clases deban conocer todos los detalles internos del sistema.

4. Justificación de cada patrón seleccionado

Factory Method / Simple Factory

Este patrón fue seleccionado porque el sistema necesita crear distintos tipos de solicitudes. En lugar de instanciar directamente Reclamo, TareaMantenimiento o IncidenteSeguridad en varias partes del código, se centraliza esa responsabilidad en SolicitudFactory. Esto mejora la organización y facilita agregar nuevos tipos de solicitudes en el futuro.

State

Este patrón fue seleccionado porque las solicitudes pueden cambiar de estado durante su ciclo de vida. Por ejemplo, una solicitud puede estar pendiente, en proceso, resuelta o cancelada. Usar clases de estado permite evitar una gran cantidad de condicionales y organizar mejor la lógica de transición.

Observer

Este patrón fue seleccionado porque el sistema necesita notificar cambios relevantes. Cuando cambia el estado de una solicitud, los usuarios interesados, como residentes o administradores, pueden recibir una actualización automáticamente. Esto reduce el acoplamiento entre la solicitud y quienes deben ser notificados.

Facade / Controller

Este patrón fue seleccionado porque el sistema necesita una clase que coordine las operaciones principales. SistemaBarrio funciona como punto de entrada para las acciones más importantes, simplificando la interacción entre los objetos y evitando que la lógica quede dispersa.

5. Identificación de principios SOLID aplicados.

Single Responsibility Principle

Cada clase tiene una responsabilidad principal. Por ejemplo, Residente representa a una persona que vive en el barrio, Visitante representa a una persona autorizada a ingresar, Solicitud representa un pedido o incidente, y Acceso representa un ingreso o egreso.

Open/Closed Principle

El sistema permite agregar nuevos tipos de solicitudes o nuevos estados sin modificar toda la estructura existente. Por ejemplo, se podría agregar una nueva clase SolicitudLimpieza heredando de Solicitud.

Liskov Substitution Principle

Las clases Reclamo, TareaMantenimiento e IncidenteSeguridad pueden utilizarse como objetos de tipo Solicitud sin romper el funcionamiento del sistema.

Interface Segregation Principle

Se utilizan interfaces específicas, como EstadoSolicitud y Observador, evitando obligar a las clases a implementar métodos que no necesitan.

Dependency Inversion Principle

El sistema depende de abstracciones como Solicitud, EstadoSolicitud y Observador, en lugar de depender únicamente de clases concretas. Esto favorece un diseño más flexible y mantenible.

6. Identificación de patrones GRASP aplicados.

Information Expert

Cada clase administra la información que le corresponde. Por ejemplo, Solicitud conoce su descripción, prioridad y estado; Acceso conoce la fecha de ingreso y egreso; Vivienda conoce los residentes asociados.

Creator

La clase SolicitudFactory se encarga de crear solicitudes, ya que centraliza la lógica necesaria para decidir qué tipo de solicitud se debe instanciar.

Controller

La clase SistemaBarrio funciona como controlador principal, ya que recibe las operaciones del sistema y coordina la interacción entre las distintas clases.

Low Coupling

El diseño busca reducir dependencias innecesarias. Por ejemplo, las notificaciones se manejan mediante la interfaz Observador, evitando que Solicitud dependa directamente de clases concretas.

High Cohesion

Cada clase tiene una función clara y relacionada con su responsabilidad. Esto permite que el sistema sea más fácil de entender, modificar y mantener.

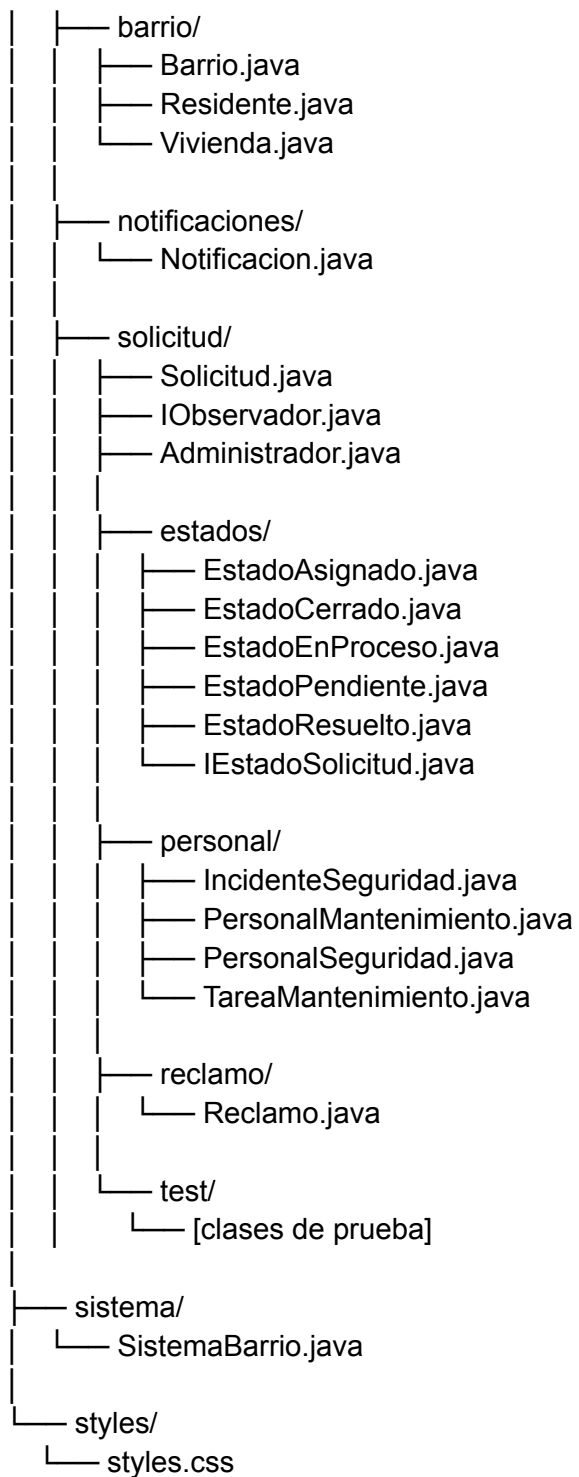
Polymorphism

El sistema utiliza polimorfismo en la jerarquía de Solicitud, permitiendo tratar de forma uniforme a Reclamo, TareaMantenimiento e IncidenteSeguridad.

7. Estructura inicial de paquetes del proyecto Java.

La estructura inicial del proyecto Java se organiza de la siguiente manera:

```
src/
├── app/
│   └── MainApp.java
├── model/
│   ├── accesos/
│   │   ├── Acceso.java
│   │   └── Visitante.java
│   └── ...
```



Esta estructura permite separar la aplicación principal, las clases del modelo, los patrones de diseño, los servicios y los recursos visuales de JavaFX.

8. Primera versión del código con clases principales, interfaces y relaciones.

En esta etapa se desarrolló una primera versión del código con las clases principales, interfaces y relaciones definidas en el diagrama de clases.

La implementación inicial incluye:

- Clases del dominio: Barrio, Vivienda, Residente, Visitante, Administrador, PersonalSeguridad, PersonalMantenimiento, Acceso y Notificacion.
- Jerarquía de solicitudes: Solicitud, Reclamo, TareaMantenimiento e IncidenteSeguridad.
- Interfaces y clases vinculadas a patrones:
 - SolicitudFactory para la creación de solicitudes.
 - EstadoSolicitud y sus estados concretos.
 - Observador para notificaciones.
 - SistemaBarrio como fachada/controlador.
- Una primera interfaz gráfica en JavaFX.
- Funcionalidad en memoria para registrar residentes y visitantes.

Esta versión no incluye persistencia en base de datos. Los datos se manejan en memoria durante la ejecución del programa.

El programa y el código fuente correspondiente a la Fase 2 se encuentran disponibles en el siguiente repositorio:

<https://github.com/sofiruba/eBarrio>

Allí se pueden visualizar los avances individuales mediante commits, la estructura del proyecto Java, las clases principales y la primera versión funcional en memoria con interfaz JavaFX.

9. Informe breve de avance individual por integrante.

Etchart, Facundo

Durante esta fase trabajó en el módulo de solicitudes y patrones de diseño. Desarrolló la jerarquía formada por Solicitud, Reclamo, TareaMantenimiento e IncidenteSeguridad. Además, implementó la estructura necesaria para aplicar el patrón State mediante EstadoSolicitud y sus estados concretos. También colaboró en la justificación de patrones de diseño y su relación con SOLID y GRASP.

Cortés, Ignacio

Durante esta fase trabajó en el módulo de accesos, seguridad, administración y notificaciones. Desarrolló clases como Acceso, Administrador, PersonalSeguridad, PersonalMantenimiento y Notificacion. También participó en la implementación de

SistemaBarrio como clase controladora/fachada y en la aplicación del patrón Observer para notificar cambios relevantes del sistema.

Rubachin, Sofía

Durante esta fase trabajó en el módulo de residentes, viviendas y visitantes. Desarrolló las clases Barrio, Vivienda, Residente y Visitante, incluyendo atributos, constructores, métodos básicos y relaciones entre las clases. Además, implementó una primera interfaz gráfica en JavaFX con la identidad visual de eBarrio, permitiendo visualizar datos en tablas y registrar residentes y visitantes en memoria. También colaboró en la documentación del módulo y en la evidencia de avance individual mediante commits en Git.